

Classify: Smart Attendance System Based on Face Recognition Integrated with Cloud and Mobile App

Kelvin Leonardo Sianipar¹, Minhajul Yusri Khairi¹, Joy Abrian Rantepasang¹, Trisman Tegar Wiratama¹, Izzata Clarissa Salsabila¹, Nurul Alya¹, MiftaHoiri Bachrudin Basir¹

¹ Information System, Department of Mathematics, Faculty of Mathematics and Natural Science, Hasanuddin University

* Correspondence: kelvin.sianipar@unhas.ac.id

† Current address: Jl. Perintis Kemerdekaan Km.10 Tamalanrea, Makassar Sulawesi Selatan Indonesia

‡ These authors contributed equally to this work.

Abstract: Student attendance systems are essential in educational management, yet traditional methods often prove time-consuming and prone to errors. This research presents the development of an automated smart attendance system based on deep learning, utilizing face recognition technology to record student attendance in real-time. The YOLOv8m model, a convolutional neural network known for its accuracy and speed in object detection tasks, including face recognition, was employed. The dataset used consists of 3,873 labeled facial images of students, captured under various lighting and image quality conditions to ensure optimal model performance in real-world environments. Evaluation results demonstrate excellent performance, with an average precision of 0.995, high recall, and an F1 score of 1.00 at a confidence level of 0.8, indicating the model's reliability in face classification with minimal misclassification. This system is expected to improve administrative efficiency by reducing the time and cost associated with manual attendance methods, while ensuring the security of students' biometric data. Overall, the implementation of deep learning in this attendance system offers a faster, more accurate solution with potential for broader application in educational institutions.

Keywords: Smart Attendance System, Deep Learning, Face Recognition, YOLOv8m

1. Introduction

Student attendance is a crucial element in educational management, serving to monitor student participation in class. Traditional attendance methods, such as manually recording signatures or calling students one by one, tend to be time-consuming and prone to high error rates. Therefore, there is an urgent need to develop a more efficient and accurate attendance system. A more advanced attendance system would facilitate educators in verifying and recording student attendance quickly and precisely.

In today's digital era, information technology is rapidly evolving and offers various innovative solutions for many fields, including education. One technology that is becoming increasingly popular and relevant is deep learning, a branch of machine learning capable of mimicking the brain's information processing patterns when handling large amounts of data and recognizing complex patterns. The application of deep learning in the development of technology-based attendance systems can be an effective solution to overcome the limitations of traditional attendance methods.

The goal of this project is to develop a smart attendance system based on deep learning, which can automatically record student attendance in real-time using face recognition technology. By leveraging machine learning algorithms, this system aims to improve the accuracy and efficiency of the attendance process. In contrast to traditional methods, the deep learning-based system not only minimizes manual errors but also reduces the time spent on recording attendance. This system has the potential to streamline administrative

Citation: . Classify: Smart Attendance System Based on Face Recognition Integrated with Cloud and Mobile App. *Sensors* **2024**, *1*, 0. <https://doi.org/>

Received:

Revised:

Accepted:

Published:

Copyright: © 2024 by the authors. Submitted to *Sensors* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

tasks and provide an easy-to-use platform for educators, eliminating the need for complex technical training.

Furthermore, this system aims to address the challenge of environmental variability, such as changes in lighting and angle, by ensuring that the face recognition model can perform reliably under diverse conditions. The ability to recognize faces in varying environments is critical for maintaining the system’s robustness and operational efficiency. The system’s ability to continuously function optimally, even under challenging conditions, marks a significant improvement over traditional attendance systems.

To ensure data privacy and security, the project will implement strong security protocols in managing biometric data, complying with relevant regulations. This will guarantee that the student information stored is protected against unauthorized access or misuse. The system’s real-time attendance recording and easy accessibility will also contribute to greater administrative efficiency, reducing the overhead associated with attendance tracking, reporting, and record-keeping.

Overall, the implementation of deep learning in this attendance system is expected to not only improve the speed and accuracy of attendance recording but also enhance overall administrative efficiency in educational institutions. This project will evaluate the impact of such technology on reducing the time and cost associated with manual attendance methods while ensuring high levels of data accuracy and privacy.

2. Methodology

This section discusses in detail how the data was collected and used in the classification model.

2.1. Data Collection

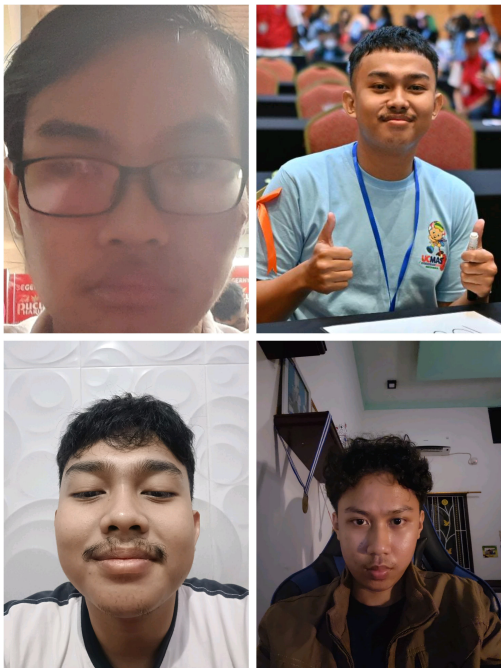


Figure 1. Example image capture

Data for the face recognition application was gathered from various sources to ensure the model’s ability to recognize faces under realistic conditions. Two main methods were used in data collection:

2.1.1. Manual Image Capture

Some facial images of team members were manually captured using a camera under various lighting conditions and backgrounds. These images were collected to ensure the dataset covered both ideal and non-ideal face conditions (see Figure 1).

2.1.2. Video Frame Extraction

In addition to manual images, video frame extraction techniques were employed. Videos were recorded in a classroom environment, and frames were extracted to increase the size of the dataset. This method allows for the inclusion of facial images from various angles, poses, and expressions.

2.1.3. Labeling Images with Roboflow

After gathering the images, we used Roboflow, a popular image annotation tool, to label the dataset. Roboflow enabled us to efficiently label each image with the corresponding student’s identity, creating a structured dataset for training the face recognition model. With Roboflow, we were able to organize the images into different classes, each representing our team member (see Figure 2).

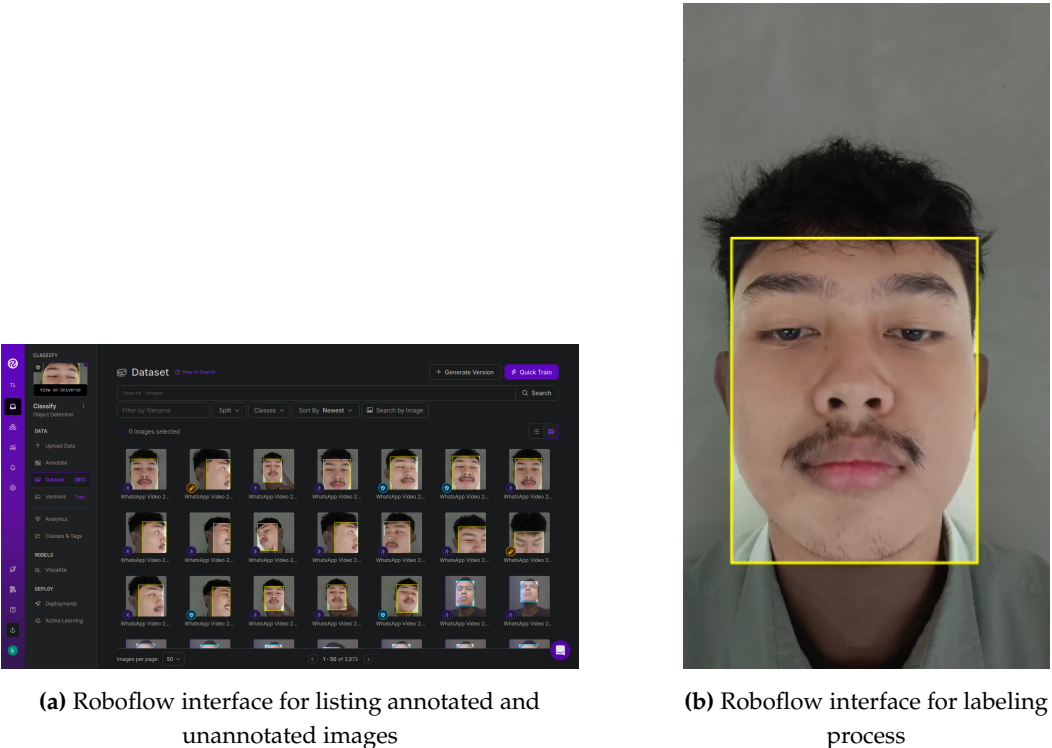


Figure 2. Roboflow interface: (a) Annotation list view, (b) Labeling process view

2.2. Data Explanation

The dataset used for the attendance system consists of facial images of students, each assigned a label corresponding to a unique student identity. The key features of the dataset are outlined as follows:

2.2.1. Images

Each image is stored in a common format such as JPEG or PNG, with varying resolutions ranging from 640x640 pixels to 720x1920 pixels. The images were captured to ensure the facial features are clear and can be accurately recognized by the model.

2.2.2. Classes and Labels

Each student in the dataset is represented as a separate class, with each image assigned to a corresponding label. There are between 638 and 661 labeled images for each team member, based on the number of images available. The labels, which were generated using Roboflow, allow the model to learn to distinguish between different team member by identifying their unique facial features.

2.2.3. Conditions of Image Capture

The images in the dataset were taken under a variety of conditions to simulate real-world classroom environments. These include low-light conditions, outdoor environments, and images taken at low resolutions. These diverse conditions ensure that the model can recognize faces in less-than-ideal situations, such as varying lighting or poor image quality.

2.2.4. Preprocessing

Once labeled, the images underwent preprocessing before being used for model training. This included resizing the images to a standard resolution and normalizing pixel values to maintain consistency. Additionally, data augmentation techniques such as random rotations and zooms were applied to enhance the dataset and make the model more robust to different face angles and expressions

2.3. Algorithm or Model

For this project, we utilized the YOLOv8 model for face recognition and detection. YOLO (You Only Look Once) is a convolutional neural network known for its accuracy and speed in object detection tasks, making it suitable for real-time applications such as face recognition. The model was trained using a dataset of labeled faces to allow it to distinguish individual team members. We used the medium variant of YOLOv8, which balances accuracy and computational efficiency, making it a good choice for our dataset size and target application.

```
import os
from ultralytics import YOLO

model = YOLO("yolov8m.pt")

train_results = model.train(data="data.yaml", epochs=100, imgsz=640)
```

2.4. Testing

2.4.1. Testing Procedures

The testing process for the Classify App focuses on validating the model's accuracy, functionality, and security compliance. Specific scenarios, such as low-light conditions, angled views, and multiple faces in a frame, are incorporated to assess the system's performance. Procedures include functional verification of features like registration, login, and attendance recognition, ensuring each works as expected across different device and environmental conditions.

2.4.2. Testing Metrics

Key testing metrics, including accuracy, precision, and compliance with security standards, are measured to assess the model's performance. Accuracy metrics gauge the model's reliability across diverse lighting and angle variations, while security tests validate data encryption during capture and storage. These metrics provide insight into both functional performance and data safety.

2.5. Evaluation Metrics

Evaluation metrics encompass recognition accuracy, processing speed, and resilience under varied conditions. Accuracy is measured to ensure an 85

3. Intelligence system

3.1. System Architecture

The system architecture in the current Classify application is designed to meet the needs of a simpler scale. This certainly needs further development to ensure that the system development plan, which has been designed from the beginning, can be properly supported by the system architecture. One example is the use of SQLite as the database to store student data. As a local data persistence solution, SQLite cannot handle online data integration, making it difficult to synchronize data when the application is used within an institutional environment. Therefore, a more robust database is required to address this issue, such as a cloud server or services like Google Firebase.

3.2. System Workflow

3.2.1. Use Case Diagram

The Classify application has two main actors: students and teachers. Both have different access levels to various system features. Here is an explanation of each stage in the use case:

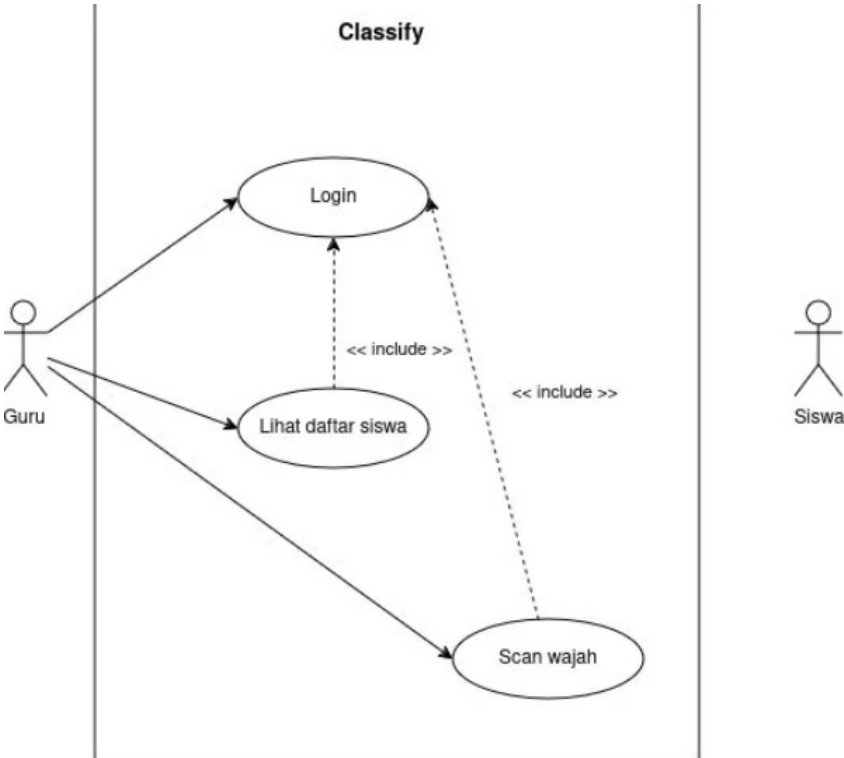


Figure 3. Use Case Diagram Classify App

- **Login:** Teachers can log in to access the features available in the application.
- **View Student List:** Teachers can view the names of students registered in the class along with the last attendance date.
- **Face Scan:** Teachers can perform a face scan of students to enable automatic attendance.

3.2.2. Activity Diagram

To clarify the workflow of the application, here are several activity diagrams that illustrate user interactions with the application, from face scanning to automatic attendance:

- **Activity Diagram - Login:** This diagram shows the workflow of the teacher performing the login process. It begins with opening the application. If a session is still active/saved, the user will automatically be directed to the main page. On the other hand, if no session exists or it has been deleted, the user will be taken to the login page, where the teacher will fill in the login form. If the data is incorrect or login fails, the user will be redirected back to the login page. However, if the login is successful, the user will be taken to the main page.

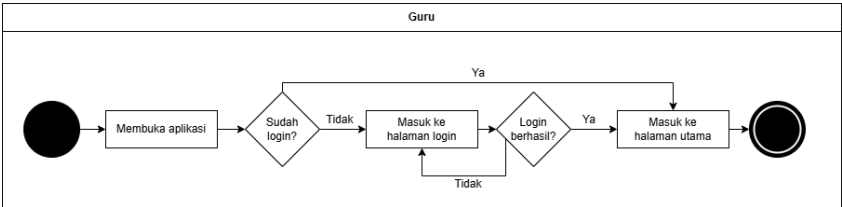


Figure 4. Activity Diagram - Automatic Attendance

- **Activity Diagram - Automatic Attendance:** This diagram shows the workflow of the automatic attendance process. It starts with the teacher uploading student face photos to the server. Once the face is successfully recognized, the system will automatically display and update the student attendance data.

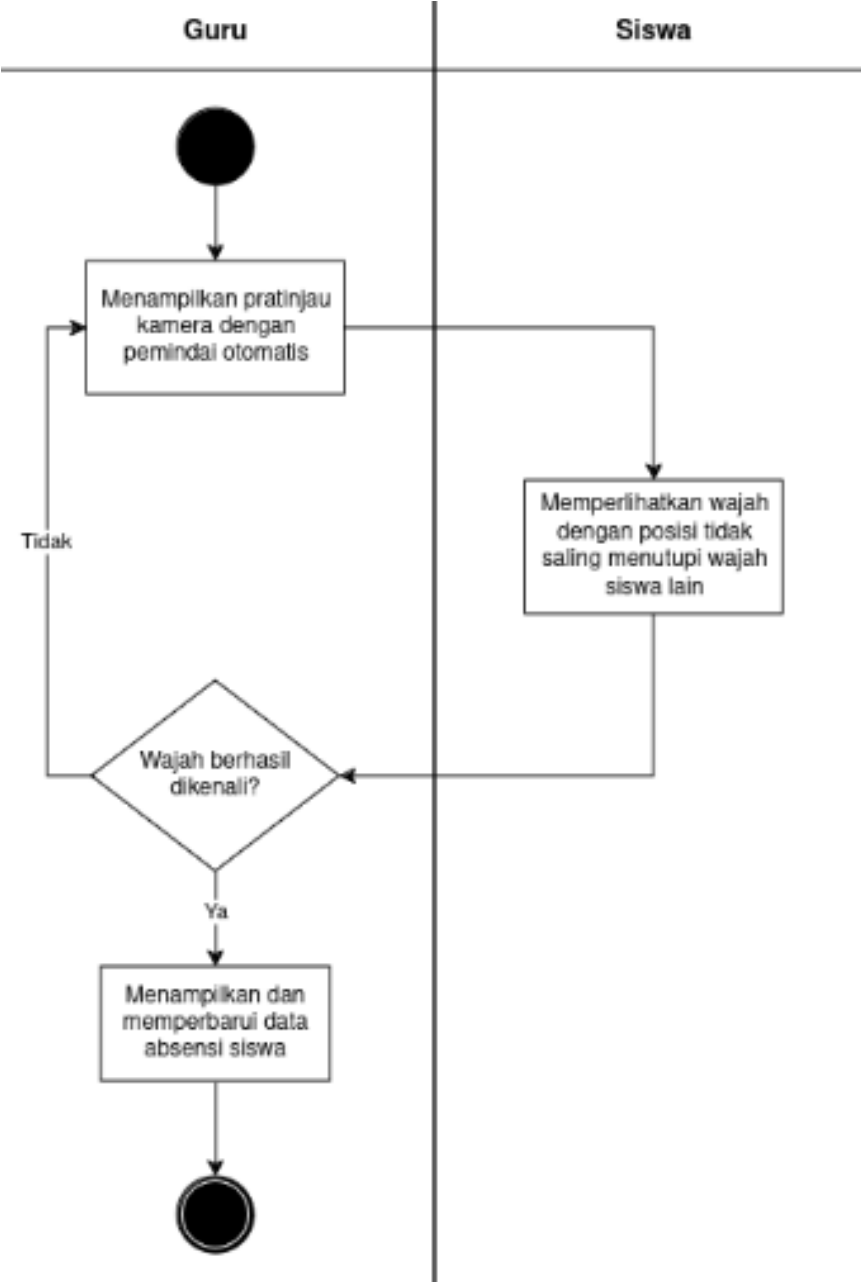


Figure 5. Activity Diagram - Automatic Attendance

- **Activity Diagram - Viewing the Student List:** This diagram shows the workflow for opening the tab to view the student list. It begins with the teacher opening the sidebar and then navigating to the student list page.

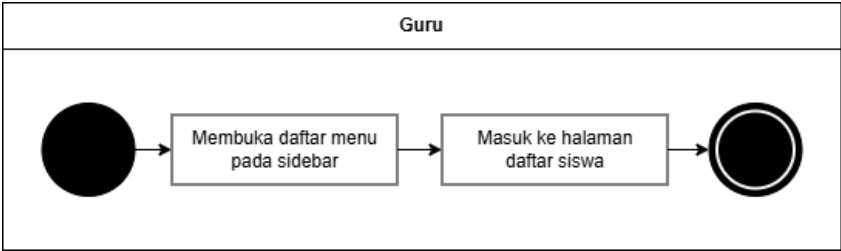


Figure 6. Activity Diagram - Viewing the Student List

- **Activity Diagram - Logout:** This diagram shows the workflow of the teacher logging out. It starts with the teacher opening the sidebar and then pressing the logout button.

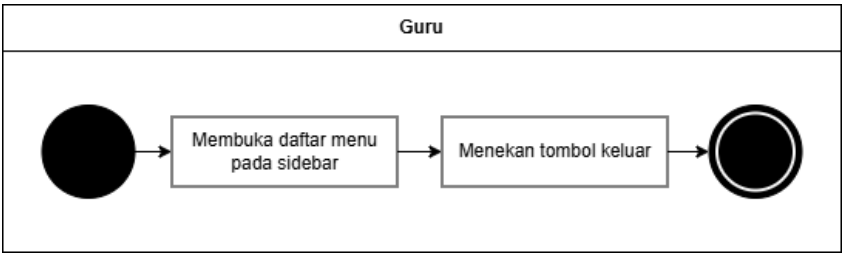


Figure 7. Activity Diagram - Logout

3.2.3. Sequence Diagram

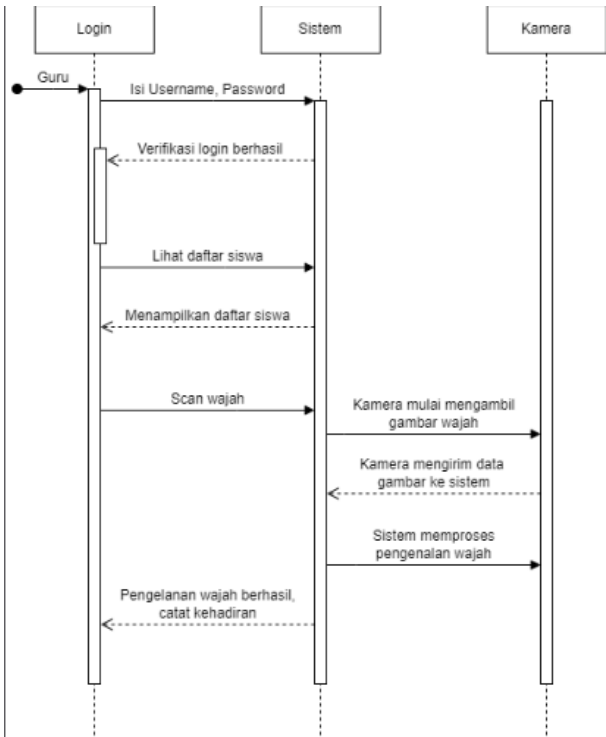


Figure 8. Sequence Diagram Classify App

The teacher logs in by entering the correct username and password. If the login verification is successful, they will be directed to the main interface. The teacher can view the list of students provided by the system. When performing a face scan, the system uses the device’s camera to capture an image of the student’s face. The system then processes face recognition, and if the recognition is successful, it updates the student’s attendance data and displays it on the user interface.

4. Results

In this section, the results obtained from training the YOLOv8m model and its performance analysis will be discussed.

4.1. Dataset and Training

- **Dataset and Annotation:** The dataset used for training consisted of 3,873 face images across six classes (Ajul, Izzata, Joy, Kelvin, Lia, and Trisman), each labeled for face detection and classification. The **YOLOv8m** model was chosen for its performance in handling diverse face detection scenarios.

- **Training Process:** The YOLOv8m model was trained using **AdamW** with a learning rate of **0.001** and momentum of **0.9** over **100 epochs**. Data augmentations, including blur and grayscale conversion, were applied to improve robustness. A validation set of **774 images** was used to evaluate the model's performance and ensure it generalized well.

4.2. Model Evaluation Metrics

The model was evaluated using key metrics such as Precision, Recall, F1-Score, and mean Average Precision (mAP).

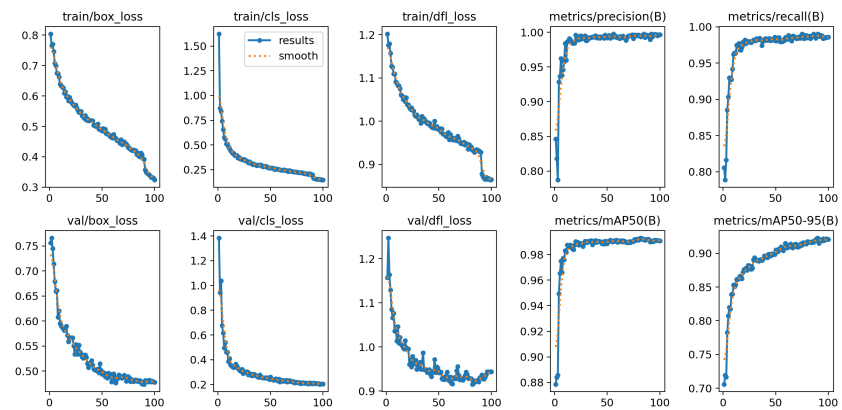


Figure 9. Performance results for YOLOv8 model across all classes

4.2.1. Precision-Recall Analysis

The **Precision-Recall curve** shows an average precision of **0.995** across all classes, indicating excellent performance in distinguishing between different classes. Each class (Ajul, Izzata, Joy, Kelvin, Lia, and Trisman) achieved a precision score of **0.995** at a high recall level, suggesting that the model can confidently classify faces with minimal misclassification.

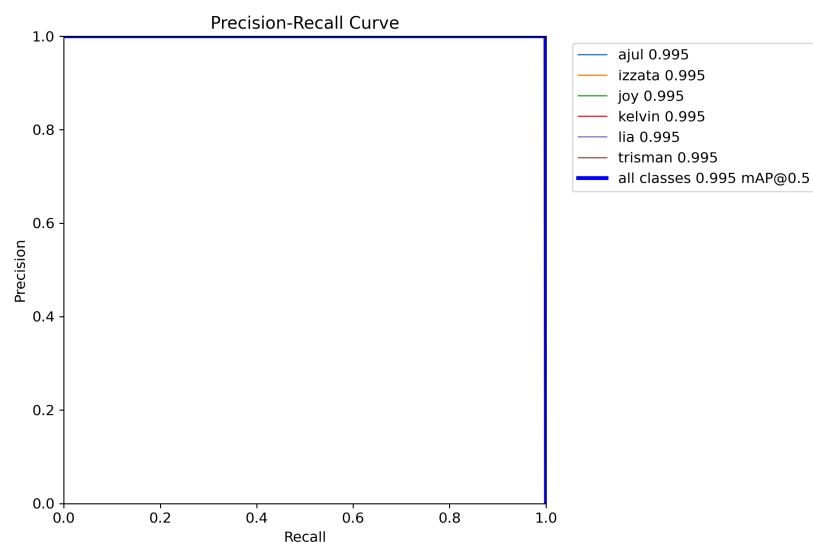


Figure 10. Precision-Recall curve for YOLOv8m model across all classes

4.2.2. F1-Confidence Analysis

The **F1-Confidence curve** indicates an F1 score of **1.00** for all classes at a confidence level of **0.8**, meaning the model is both highly precise and accurate in predictions. As confidence increases towards **1.0**, the model maintains a high F1 score across all classes, further affirming its reliable classification performance.

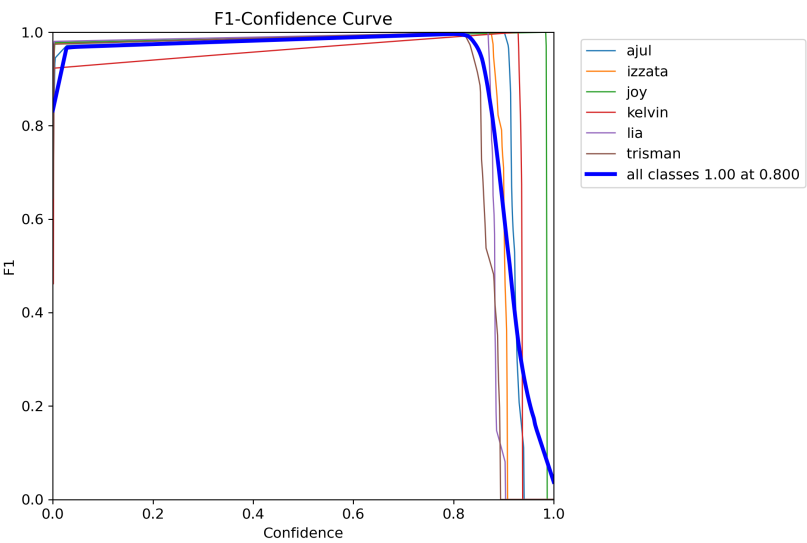


Figure 11. F1 curve for YOLOv8m model across all classes

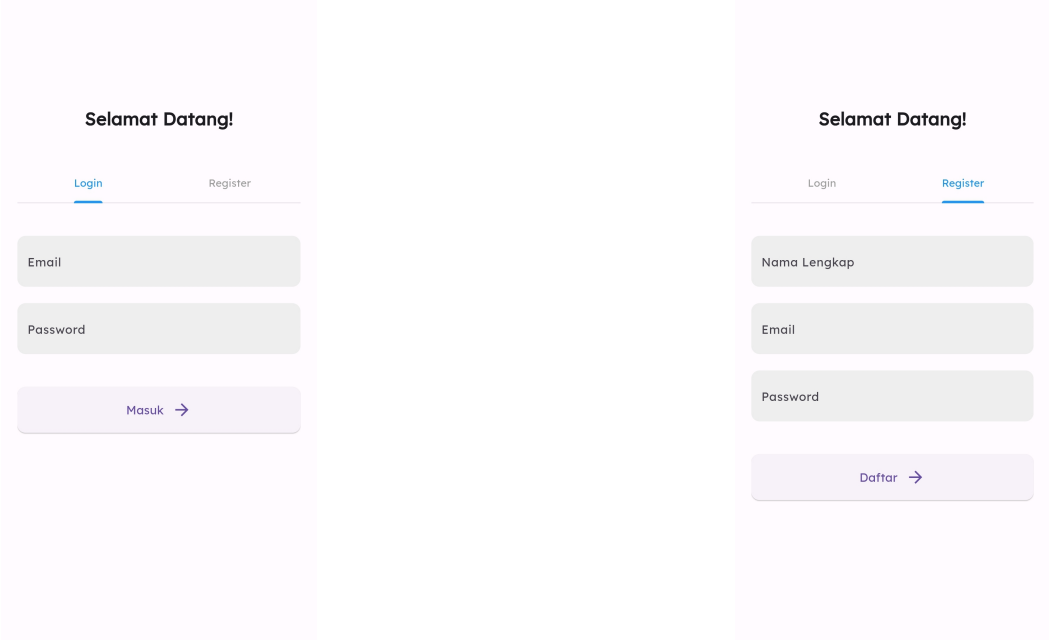
4.2.3. Performance Summary205

The YOLOv8m model achieved an overall mean Average Precision (mAP@0.5) of206
0.995, demonstrating its effectiveness in the face recognition task. These results suggest207
that the model is suitable for applications requiring high accuracy in face classification,208
such as attendance systems or security monitoring.209

4.3. Application Development210

4.3.1. Frontend211

• Login and Register212



(a) Login Interface

(b) Register Interface

Figure 12. Login and Register Interface

• Main Page213

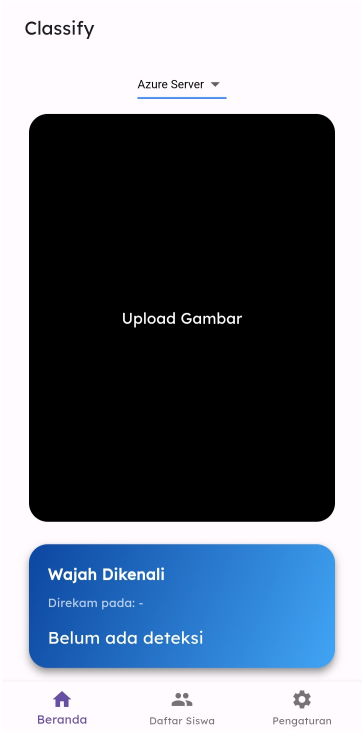


Figure 13. Main Page Interface

- Student List

214

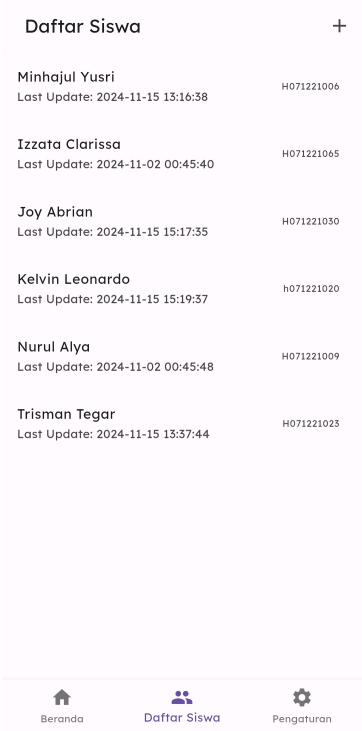


Figure 14. Student List Interface

- Add New Data

215

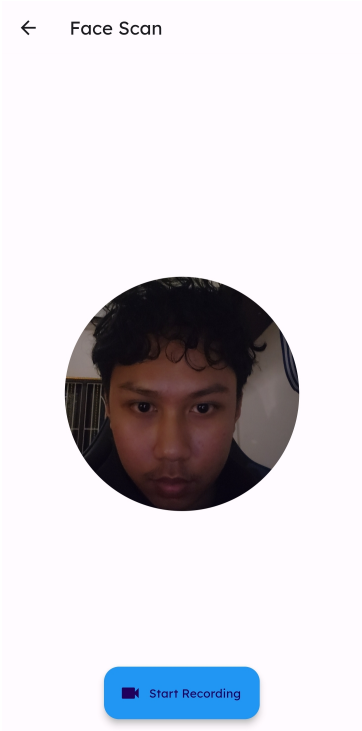


Figure 15. Add New Data

- **Profile**

216

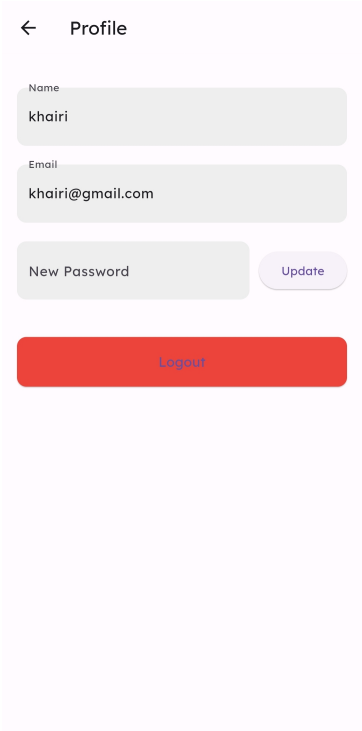


Figure 16. Profile

4.3.2. Backend

217

The following backend components have been completed:

218

- **API for receiving and processing video:** This code has the main function of receiving, storing, processing, and extracting frames from a video file uploaded via the API.

219

220

```

@app.route('/save-video', methods=['POST'])
def save_video():
    if 'video' not in request.files:
        return jsonify({"error": "No file part"}), 400

    file = request.files['video']
    if file.filename == '':
        return jsonify({"error": "No selected file"}), 400

    # Save the video temporarily
    video_path = os.path.join("temp", file.filename)
    os.makedirs("temp", exist_ok=True)
    file.save(video_path)

    # Generate a unique folder name based on the current date and time
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_folder = os.path.join("extracted", f"frames_{timestamp}")
    os.makedirs(output_folder, exist_ok=True)

    # Convert video to frames
    frame_count = convert_video_to_frames(video_path, output_folder)

    # Remove the temporary video file
    os.remove(video_path)

    return jsonify({
        "message": "Video successfully saved and frames extracted",
        "frames_count": frame_count,
        "output_folder": output_folder
    })

```

- **Extract Frames from Video with OpenCV:** This code is a Python function for extracting each frame from a video and saving it as an image file.

```

def convert_video_to_frames(video_path, output_folder):
    cap = cv2.VideoCapture(video_path)
    frame_count = 0

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        # Save each frame as an image in the output folder
        frame_filename = os.path.join(output_folder, f"frame_{frame_count}")
        cv2.imwrite(frame_filename, frame)
        frame_count += 1

    cap.release()
    return frame_count

```

- **YOLO Object Detection API:** This code provides an API endpoint to detect objects using the YOLO model.

```

model = YOLO("terbaru.pt")

@app.route('/detect', methods=['POST'])
def detect_objects():
    if 'image' not in request.files:
        return jsonify({"error": "No file part"}), 400

    file = request.files['image']

    # Check if the file is valid
    if file.filename == '':
        return jsonify({"error": "No selected file"}), 400

    img_array = np.frombuffer(file.read(), np.uint8)
    image = cv2.imdecode(img_array, cv2.IMREAD_COLOR)

    # Inference on the image
    results = model.predict(source=image, device="cpu", conf=0.5, iou=0.4,

    detected_classes = []

    for result in results:
        for box in result.bboxes:
            cls = int(box.cls.item()) # Get class index and convert to int
            class_name = result.names[cls] # Get the class name from the
            if class_name not in detected_classes:
                detected_classes.append(class_name) # Add class name to the list

    return jsonify({"detected_classes": detected_classes})

```

5. Conclusion

The development of a deep learning-based attendance system using facial recognition technology, as implemented in the Classify application, offers a significant solution to improve the efficiency and accuracy of student attendance recording. This system is capable of overcoming the limitations of traditional attendance methods, which tend to be time-consuming and prone to errors.

The high precision, recall, and F1 scores across all classes reflect the YOLOv8m model's strong capability in recognizing and distinguishing between the specified faces. This level of performance underscores the model's potential for real-world deployment in face recognition tasks with high accuracy and reliability.

References

1. Anwar, S.; Setiawan, H. Pengenalan Wajah Menggunakan Metode Convolutional Neural Network (CNN) pada Sistem Absensi. *Jurnal Teknologi Informasi dan Ilmu Komputer* **2018**, *5*(2), 134–141.
2. Sudarma, M.; Purnama, I. K. Pengantar Machine Learning dan Deep Learning; Andi Offset: Yogyakarta, 2020.
3. Anggraini, W. Deep Learning Untuk Deteksi Wajah Yang Berhijab Menggunakan Algoritma Convolutional Neural Network (CNN) Dengan Tensorflow; Doctoral Dissertation, UIN Ar-Raniry Banda Aceh, 2020.
4. Wibowo, T.; Hadi, W. Implementasi Teknologi Pengenalan Wajah dalam Sistem Absensi Berbasis Deep Learning. *Jurnal Sistem Informasi* **2021**, *10*(1), 89–102.
5. Dewi, N.; Ismawan, F. Implementasi deep learning menggunakan CNN untuk sistem pengenalan wajah. *Faktor Exacta* **2021**, *14*(1), 34–43.
6. Zhou, S.; Chellappa, R.; Zhao, W. *Unconstrained face recognition in Machine Learning*; Springer Science & Business Media: Berlin, 2006.

-
7. Ahmad, A. Mengenal artificial intelligence, machine learning, neural network, dan deep learning. *J. Teknol. Indones.* **2017**, *October*, 325
3.